

Interactive Security Training Platform

Project Outline – Software-Projekt 4

IT.PM4, Spring Semester 2026

From Reading to Exploiting
Hands-on Security Training for Universities

Submission Date:	15 March 2026
Module:	Software-Projekt 4 (IT.PM4)
Institution:	ZHAW – School of Engineering
Team:	Biedermann Linus, Calabrese Davide, Hoffmann Lorenz, Kaiser Jan, Schaub Alex, Seeberger Alessio

Table of Contents

1	Initial Situation & Idea	3
1.1	Problem Statement	3
1.2	Solution Idea: Interactive Security Training Platform	3
1.3	Stakeholder Analysis	3
2	State of the Art / Competitive Analysis	4
3	Context Scenario (Main Flow)	5
4	Further Requirements	6
4.1	Planned Extensions after PM4	6
4.2	Security and Maintenance	6
5	Resources and Timeline	7
5.1	Team Competencies	7
5.2	Technology Stack	7
5.3	Timeline / Roadmap	8
6	Risks	8
7	Economic Viability	10
8	Outlook	10
8.1	Potential Adoption at ZHAW	10
8.2	NoTech Deliverable: Promotional Video	10
9	Glossary	12
	References	14

1 Initial Situation & Idea

1.1 Problem Statement

Application security is a central topic in modern software development. Yet traditional courses often teach security concepts purely in theory: students read about vulnerabilities like SQL injection or cross-site scripting without ever actively exploiting them in a safe environment [1]. This gap between theory and practice means graduates can name vulnerability classes but struggle to recognise or fix them in real code.

Platforms like *TryHackMe* or *HackTheBox* exist, but run on external cloud infrastructure and give universities no control over content, data privacy, or grade integration.

1.2 Solution Idea: Interactive Security Training Platform

The **Interactive Security Training Platform** (abbreviated as ISTP) is a self-hosted, Kubernetes-based Capture-the-Flag learning platform that universities can operate on-premises. Students receive isolated, ephemeral container environments in which they actively exploit real vulnerabilities instead of just reading about them. Instructors prepare challenges in advance so students can apply what they learned in class directly in practice.

1.3 Stakeholder Analysis

Table 1 discusses how exactly ISTP is going to benefit the stakeholders of this project.

The column “Need / Problem” discusses what needs the specific stakeholder has in regard to ISTP and what problems they face when using currently established tools on the market. Those tools meet their needs only partially. This leads to their needs and problems overlapping. Due to this overlap, the needs and problems of the stakeholders have been placed into the same column.

The column “Benefits from ISTP” describes which benefits the stakeholders receive by using ISTP and how their problems get solved.

Stakeholder	Need / Problem	Benefits from ISTP
Students	They need hands-on exercises in application security. The exercises need to match the lectures they attended. They do not want to spend money on the exercises.	They receive active exploit training in a safe sandbox. The exercises are created by the instructors or TAs. Therefor they are specifically designed for the lectures. Because ISTP is hosted by the university, these exercises cost the students no money.
Instructors / TAs	They require a simple management of the exercises. They do not want to spend money on this task.	Using an instructor panel, they can create challenges and manage their lifecycle. Because ISTP is hosted by the university, the management of the exercises cost the instructors and TAs no money.
IT Administration	They need privacy-compliant and maintainable infrastructure.	They receive an application that they can deploy on-premises on the university's own Kubernetes cluster. Now they have full control over the privacy policy and maintenance of the training platform.

Table 1: Stakeholders and their benefits

As seen in Table 1, ISTP meets the needs of the stakeholders and solves their problems with established tools.

2 State of the Art / Competitive Analysis

Several platforms exist for hacking exercises [2], [3], [4]. Table 2 summarises the most relevant alternatives.

Platform	Hosting	Focus	Suitable for Academia?
TryHackMe [2]	Cloud (external)	Broad spectrum	Limited (paid tiers, no custom challenges [5])
HackTheBox [3]	Cloud (external)	Advanced CTFs	No (advanced level only, no custom challenges, paid features)
PentesterLab [4]	Cloud (external)	Web vulnerabilities	Partially (many exercises locked behind paywall [6])
ISTP	On-Premises	Application security (customisable)	Yes, built for academia

Table 2: Competitive comparison

ISTP is the only solution that runs fully on-premises, offers instructor-controlled application security challenges, and integrates with academic workflows including role management and challenge lifecycle.

The fact that ISTP can run fully on-premises means that neither instructors nor students need to pay for using ISTP. This makes ISTP from a financial point of view, in addition to the already stated advantages, more suited for educational usage. Currently the ZHAW uses TryHackMe in the module “IT Security”. But as stated in Table 2, TryHackMe has paid tiers. When visiting “IT Security” we have noticed that we can only use TryHackMe for one hour per day. Once that hour has passed, many features get locked behind a paywall.

3 Context Scenario (Main Flow)

The following scenario describes the typical use case from both the student and instructor perspectives. The perspective of the IT administrator is not important for the typical use cases of this application. Administrators have additional permissions and can, for example, ban a user.

Scenario: «Solving a first CTF challenge»

A student at the university registers on the ISTP platform using her university email address. After logging, in she is able to see all published challenges (e.g., SQL injection, XSS, IDOR).

She selects a challenge and launches a dedicated pod with one click. The system provisions an isolated Kubernetes pod hosting the vulnerable application. The application then gets automatically opened directly in her browser.

She reads the task description and analyses the application, identifies the vulnerability and exploits it to extract a hidden flag (e.g., `FLAG{sql_injection_mastered}`). She submits the flag via the UI. The system validates the input, updates her score and marks the challenge as solved.

After 60 minutes of inactivity, a keep-alive prompt appears. If the student does not confirm it, the pod terminates automatically to free cluster resources.

Instructor perspective: An instructor creates a new challenge by entering a container image, a task description and the expected flags in the admin panel. After making some configurations, she publishes the challenge, which immediately becomes visible and launchable for all students.

4 Further Requirements

4.1 Planned Extensions after PM4

- **SSO integration:** Connection to the university LDAP/SAML infrastructure for single sign-on.
- **Assignments:** Instructors can assign exercises to students and set a deadline.
- **Grade integration:** The application can automatically grade students based on their assigned exercises. Instructors can export the grades of their students.
- **Extended scoring:** Time bonuses and hint penalties for competitive use.
- **Leaderboard:** Public ranking per course or semester.
- **Challenge marketplace:** Sharing challenges between universities.
- **Multi-cluster support:** Scaling across multiple Kubernetes clusters.

4.2 Security and Maintenance

Since students actively exploit vulnerabilities, resource limits per pod and network policies are mandatory. Regular updates of all components are planned as a fixed part of ongoing operations.

5 Resources and Timeline

5.1 Team Competencies

Person	Skills	Focus
Biedermann Linus	Documentation, Infrastructure, Java, React	Documentation, Fullstack Dev
Calabrese Davide	Java, REST, React, Database	Fullstack Dev
Hoffmann Lorenz	Java, React, SpringBoot, Keycloak	Security, Fullstack Dev
Kaiser Jan	Java, React, Cypress, Database	Scrum Master, Fullstack Dev, QA
Schaub Alex	Java, React, SpringBoot, Database	Fullstack Dev, Documentation
Seeberger Alessio	Infrastructure, Kubernetes, React, CI/CD	Product Owner, Fullstack Dev, DevOps

Table 3: Team competencies and responsibilities

5.2 Technology Stack

Table 4 lists the technologies that will be used for each layer and why they have been chosen.

Layer	Technology	Justification
Frontend	Next.js 16+	Server-side rendering framework
Backend	Spring Boot 4.0.3	Robust REST framework with Keycloak integration
Authentication	Keycloak 26+	Role and token management out of the box
Orchestration	Kubernetes	Can run isolated pods for each exercise and enforces resource quotas
Database	PostgreSQL	Reliable relational DB with full JPA support
CI/CD	GitHub Actions	Automated lint, test and build checks

Table 4: Technology stack

5.3 Timeline / Roadmap

Phase	Period (CW)	Milestone
Sprint 1	CW 10-12	Authentication
Sprint 2	CW 12-14	Course management
Sprint 3	CW 14-16	Challenge creation
Sprint 4	CW 16-18	Playable Challenges
Sprint 5	CW 18-20	Statistics
Project deadline	24.05.2026	Project submission

Table 5: Roadmap with milestones

Each sprint closes with a demo and retrospective. Unfinished work is carried over to the next sprint.

6 Risks

The following seven risks in Table 6 have been identified. The risks are not ordered.

Risk	Category	Description
Authentication integration	Technical	Keycloak needs to be set up for single sign-on and JSON web tokens need to be propagated between the frontend and backend. This might be complicated.
Connecting to pods	Technical / Security	To connect the students to their pods, a secure reverse-proxy or WebSocket tunnel is needed. This is complicated and might also be a security risk.
Security of ISTP	Security	Students may try to escape their container. This is a security risk.
Resource exhaustion	Infrastructure	Due to the fact that many pods need to run in parallel, the resources of the server may get exhausted.
Kubernetes manifest validation	Security / Stability	Instructors or TAs might upload malicious or malformed container images for the exercises.
Knowledge gaps	Team	Members of the team might lack experience with parts of the technology stack.
Role management complexity	Product	Managing multiple roles such as students, instructors, and admins with different permissions can become complex and may lead to misconfigurations or unintended access rights.

Table 6: Roadmap with milestones

7 Economic Viability

ISTP is not a commissioned project from an industry partner. It grew out of a concrete experience during our own studies: we worked with external platforms where instructors had to manually check each student’s laptop to verify progress. Integrated grading was not possible.

Since the code is hosted on GitHub, ISTP will be released as an open-source solution. Universities save on licensing costs, keep full control over content and data privacy and can tailor challenges to their own learning objectives. Integrated grading is planned as a future extension.

A single instance is sufficient for multiple courses and semesters. In the medium term, adoption at other Swiss universities is realistic since the problem is not unique to ZHAW.

8 Outlook

ISTP is an open-source project that originated from a concrete gap we experienced during our own studies. There is no commercial roadmap, no crowdfunding campaign, and no plan to found a start-up. Our goal at the end of PM4 is to deliver a working, well-documented platform that any university can pick up, deploy, and use without friction.

8.1 Potential Adoption at ZHAW

If the product meets the quality bar we have set ourselves — stable, secure, and genuinely useful in a classroom context — we intend to approach the instructors responsible for the ZHAW module *IT Security* to discuss a potential pilot. Any such conversation would be informal and exploratory; adoption would be entirely at ZHAW’s discretion and subject to their own technical and compliance standards.

8.2 NoTech Deliverable: Promotional Video

The primary non-technical deliverable at the end of PM4 is a short promotional video. The format is a founder-style talking-head piece — intimate, direct, and deliberately simple — in the tradition of the early pitch videos produced by companies like Dropbox and Airbnb when they first presented their ideas.

The video will cover three things: the problem (security education that stays theoretical), the solution (hands-on, self-hosted CTF challenges that mirror real

vulnerabilities), and the value proposition for universities (free, on-premises, privacy-compliant, and built for academic workflows). It is aimed at instructors and IT administrators who might encounter ISTP — either at ZHAW or elsewhere — and need a fast, human way to understand what the platform is and whether it is worth a closer look.

9 Glossary

CTF	Capture-the-Flag — a competition format in which participants solve security challenges to retrieve hidden strings called flags.
IDOR	Insecure Direct Object Reference — an access-control vulnerability in which an attacker can access resources belonging to another user by manipulating object identifiers in requests.
ISTP	Interactive Security Training Platform — the self-hosted, Kubernetes-based learning platform developed in this project.
JWT	JSON Web Token — a compact, URL-safe token format used to transmit authentication and authorisation claims between frontend and backend.
Keycloak	An open-source identity and access-management solution that provides single sign-on, OAuth 2.0 and OpenID Connect support.
Kubernetes	An open-source container-orchestration system that automates deployment, scaling and resource management of containerised workloads.
LDAP	Lightweight Directory Access Protocol — a protocol for accessing and maintaining distributed directory information, commonly used for university identity management.
OWASP	Open Web Application Security Project — a non-profit foundation that publishes the OWASP Top 10, a widely referenced list of critical web-application security risks.
Pod	The smallest deployable unit in Kubernetes; in ISTP each student receives an ephemeral pod containing the vulnerable application for a challenge.
SAML	Security Assertion Markup Language — an XML-based standard for exchanging authentication and authorisation data, used for federated single sign-on.
SQL Injection	A code-injection attack in which malicious SQL statements are inserted into an input field, allowing an attacker to manipulate or read a database.

SSO	Single Sign-On — an authentication scheme that allows a user to log in once and gain access to multiple applications without re-entering credentials.
XSS	Cross-Site Scripting — a vulnerability in which an attacker injects malicious client-side scripts into a web page viewed by other users.

References

- [1] OWASP Foundation, “OWASP Top Ten 2021.” Accessed: Mar. 04, 2026. [Online]. Available: <https://owasp.org/Top10/>
- [2] TryHackMe, “TryHackMe – Learn Cybersecurity.” Accessed: Mar. 04, 2026. [Online]. Available: <https://tryhackme.com/>
- [3] Hack The Box, “Hack The Box – Hacking Training Platform.” Accessed: Mar. 04, 2026. [Online]. Available: <https://www.hackthebox.com/>
- [4] PentesterLab, “PentesterLab – Learn Web Penetration Testing.” Accessed: Mar. 04, 2026. [Online]. Available: <https://pentesterlab.com/>
- [5] TryHackMe, “TryHackMe | Cyber Security Training.” Accessed: Mar. 07, 2026. [Online]. Available: <https://tryhackme.com/pricing>
- [6] PentesterLab, “Penetration Testing, Security Code Review and Web App Security Exercises.” Accessed: Mar. 07, 2026. [Online]. Available: <https://pentesterlab.com/exercises>